

STEM BUILDERS



Dictionaries

7. Dictionaries

Dictionaries allow us to store connected bits of information. For example, you might store a person's name and age together.

What are dictionaries?

Dictionaries are a way to store information that is connected in some way. Dictionaries store information in *key-value* pairs, so that any one piece of information in a dictionary is connected to at least one other piece of information.

Dictionaries do not store their information in any particular order, so you may not get your information back in the same order you entered it.

General Syntax

A general dictionary in Python looks something like this:

```
dictionary_name = {key_1: value_1, key_2: value_2, key_3: value_3}
```

Since the keys and values in dictionaries can be long, we often write just one key-value pair on a line. You might see dictionaries that look more like this:

```
dictionary_name = {key_1: value_1,  
                    key_2: value_2,  
                    key_3: value_3,  
                    }
```

This is a bit easier to read, especially if the values are long.

Example

A simple example involves modeling an actual dictionary.

```
python_words = {  
    'list': 'A collection of values that are not connected, but have an order.',  
    'dictionary': 'A collection of key-value pairs.',  
    'function': 'A named set of instructions that defines a set of actions in Python',  
}
```

We can get individual items out of the dictionary, by giving the dictionary's name, and the key in square brackets:

```
python_words = {'list': 'A collection of values that are not connected, but have an order.',
                'dictionary': 'A collection of key-value pairs.',
                'function': 'A named set of instructions that defines a set of actions in Python',
                }
print("\nWord: %s" % 'list')
print("Meaning: %s" % python_words['list'])

print("\nWord: %s" % 'dictionary')
print("Meaning: %s" % python_words['dictionary'])

print("\nWord: %s" % 'function')
print("Meaning: %s" % python_words['function'])
```

Output:

```
Word: list
Meaning: A collection of values that are not connected, but have an order.
```

```
Word: dictionary
Meaning: A collection of key-value pairs.
```

```
Word: function
Meaning: A named set of instructions that defines a set of actions in Python.
```

This code looks pretty repetitive, and it is. Dictionaries have their own for-loop syntax, but since there are two kinds of information in dictionaries, the structure is a bit more complicated than it is for lists. Here is how to use a for loop with a dictionary:

```
python_words = {'list': 'A collection of values that are not connected, but have an order.',
                'dictionary': 'A collection of key-value pairs.',
                'function': 'A named set of instructions that defines a set of actions in Python',
                }
# Print out the items in the dictionary.
for word, meaning in python_words.items():
    print("\nWord: %s" % word)
    print("Meaning: %s" % meaning)
```

Output:

```
Word: function
Meaning: A named set of instructions that defines a set of actions in Python.
```

```
Word: list
Meaning: A collection of values that are not connected, but have an order.
```

Word: dictionary

Meaning: A collection of key-value pairs.

The output is identical, but we did it in 3 lines instead of 6. If we had 100 terms in our dictionary, we would still be able to print them out with just 3 lines.

The only tricky part about using for loops with dictionaries is figuring out what to call those first two variables. The general syntax for this for loop is:

```
for key_name, value_name in dictionary_name.items():  
    print(key_name) # The key is stored in whatever you called the first variable.  
    print(value_name) # The value associated with that key is stored in your second variable.
```

Exercises

- Write a Python program to check whether a given key already exists in a dictionary.
- Write a Python program to iterate over dictionaries using for loops. Print keys and values of dictionary
- Write a Python script to print a dictionary where the keys are numbers between 1 and 15 (both included) and the values are square of keys.
- Write a Python program to multiply all the items in a dictionary

Common operations with dictionaries

There are a few common things you will want to do with dictionaries. These include adding new key-value pairs, modifying information in the dictionary, and removing items from dictionaries.

Adding new key-value pairs

To add a new key-value pair, you give the dictionary name followed by the new key in square brackets, and set that equal to the new value. We will show this by starting with an empty dictionary, and re-creating the dictionary from the example above.

```
# Create an empty dictionary.
python_words = {}
# Fill the dictionary, pair by pair.
python_words['list'] = 'A collection of values that are not connected, but have an order.'
python_words['dictionary'] = 'A collection of key-value pairs.'
python_words['function'] = 'A named set of instructions that defines a set of actions in Python.'
```



```
# Print out the items in the dictionary.
for word, meaning in python_words.items():
    print("\nWord: %s" % word)
    print("Meaning: %s" % meaning)
```

Output:

Word: function
Meaning: A named set of instructions that defines a set of actions in Python.

Word: list
Meaning: A collection of values that are not connected, but have an order.

Word: dictionary
Meaning: A collection of key-value pairs.

Modifying values in a dictionary

At some point you may want to modify one of the values in your dictionary. Modifying a value in a dictionary is pretty similar to modifying an element in a list. You give the name of the dictionary and then the key in square brackets, and set that equal to the new value.

```
python_words = {'list': 'A collection of values that are not connected, but have an order.',
                'dictionary': 'A collection of key-value pairs.',
                'function': 'A named set of instructions that defines a set of actions in Python',
                }

print('dictionary: ' + python_words['dictionary'])

# Clarify one of the meanings.
python_words['dictionary'] = 'A collection of key-value pairs. Each key can be used to access its corresponding value.'

print('\ndictionary: ' + python_words['dictionary'])
```

Output:

dictionary: A collection of key-value pairs.

dictionary: A collection of key-value pairs. Each key can be used to access its corresponding value.

Removing key-value pairs

You may want to remove some key-value pairs from one of your dictionaries at some point. You can do this using the same `del` command you learned to use with lists. To remove a key-value pair, you give the `del` command, followed by the name of the dictionary, with the key that you want to delete. This removes the key and the value as a pair.

```
python_words = {'list': 'A collection of values that are not connected, but have an order.',
                'dictionary': 'A collection of key-value pairs.',
                'function': 'A named set of instructions that defines a set of actions in Python.'
                }

# Show the current set of words and meanings.
print("\n\nThese are the Python words I know:")
for word, meaning in python_words.items():
```

```
print("\nWord: %s" % word)
print("Meaning: %s" % meaning)

# Remove the word 'list' and its meaning.
del python_words['list']

# Show the current set of words and meanings.
print("\n\nThese are the Python words I know:")
for word, meaning in python_words.items():
    print("\nWord: %s" % word)
    print("Meaning: %s" % meaning)
```

Output:

These are the Python words I know:

Word: function
Meaning: A named set of instructions that defines a set of actions in Python.

Word: list
Meaning: A collection of values that are not connected, but have an order.

Word: dictionary
Meaning: A collection of key-value pairs.

These are the Python words I know:

Word: function
Meaning: A named set of instructions that defines a set of actions in Python.

Word: dictionary
Meaning: A collection of key-value pairs.

If you were going to work with this code, you would certainly want to put the code for displaying the dictionary into a function. Let's see what this looks like:

```
def show_words_meanings(python_words):
    # This function takes in a dictionary of python words and meanings,
    # and prints out each word with its meaning.
    print("\n\nThese are the Python words I know:")
    for word, meaning in python_words.items():
        print("\nWord: %s" % word)
        print("Meaning: %s" % meaning)
```

```
python_words = {'list': 'A collection of values that are not connected, but have an order.',  
                'dictionary': 'A collection of key-value pairs.',  
                'function': 'A named set of instructions that defines a set of actions in Python.',  
                }
```

```
show_words_meanings(python_words)
```

```
# Remove the word 'list' and its meaning.
```

```
del python_words['list']
```

```
show_words_meanings(python_words)
```

Output:

These are the Python words I know:

Word: function

Meaning: A named set of instructions that defines a set of actions in Python.

Word: list

Meaning: A collection of values that are not connected, but have an order.

Word: dictionary

Meaning: A collection of key-value pairs.

These are the Python words I know:

Word: function

Meaning: A named set of instructions that defines a set of actions in Python.

Word: dictionary

Meaning: A collection of key-value pairs.

As long as we have a nice clean function to work with, let's clean up our output a little:

```
def show_words_meanings(python_words):  
    # This function takes in a dictionary of python words and meanings,  
    # and prints out each word with its meaning.  
    print("\n\nThese are the Python words I know:")  
    for word, meaning in python_words.items():  
        print("\n%s: %s" % (word, meaning))
```

```
python_words = {'list': 'A collection of values that are not connected, but have an order.',  
                'dictionary': 'A collection of key-value pairs.',  
                'function': 'A named set of instructions that defines a set of actions in Python.',  
                }
```

```
show_words_meanings(python_words)
```



```
# Remove the word 'list' and its meaning.
```

```
del python_words['list']
```

```
show_words_meanings(python_words)
```

Output:

These are the Python words I know:

function: A named set of instructions that defines a set of actions in Python.

list: A collection of values that are not connected, but have an order.

dictionary: A collection of key-value pairs.

These are the Python words I know:

function: A named set of instructions that defines a set of actions in Python.

dictionary: A collection of key-value pairs.

This is much more realistic code.

Modifying keys in a dictionary

Modifying a value in a dictionary was straightforward, because nothing else depends on the value. Modifying a key is a little harder, because each key is used to unlock a value. We can change a key in two steps:

- Make a new key, and copy the value to the new key.
- Delete the old key, which also deletes the old value.

Here's what this looks like. We will use a dictionary with just one key-value pair, to keep things simple.

```
# We have a spelling mistake!
```

```
python_words = {'lisst': 'A collection of values that are not connected, but have an orde  
r.'}
```

```
# Create a new, correct key, and connect it to the old value.
```

```
# Then delete the old key.
```

```
python_words['list'] = python_words['lisst']
```

```
del python_words['lisst']
```

```
# Print the dictionary, to show that the key has changed.
```

```
print(python_words)
```

Output:

```
{'list': 'A collection of values that are not connected, but have an order.'}
```

Exercises

- Write a Python script to add a key to a dictionary.
Sample Dictionary : {0: 10, 1: 20}
Expected Result : {0: 10, 1: 20, 2: 30}
- Write a Python program to remove a key from a dictionary.
Sample Dictionary : {'c': 3, 'b': 2, 'd': 4, 'a': 1}
Expected result: {'c': 3, 'b': 2, 'd': 4}
- Write a Python program to check a dictionary is empty or not
Hint: use 'bool' function to check. 'bool' returns true if dictionary is empty

Looping through a dictionary

Since dictionaries are really about connecting bits of information, you will often use them in the ways described above, where you add key-value pairs whenever you receive some new information, and then you retrieve the key-value pairs that you care about. Sometimes, however, you will want to loop through the entire dictionary. There are several ways to do this:

- You can loop through all key-value pairs;
- You can loop through the keys, and pull out the values for any keys that you care about;
- You can loop through the values.

Looping through all key-value pairs

This is the kind of loop that was shown in the first example. Here's what this loop looks like, in a general format:

```
my_dict = {'key_1': 'value_1',  
          'key_2': 'value_2',  
          'key_3': 'value_3',  
          }  
  
for key, value in my_dict.items():  
    print('\nKey: %s' % key)  
    print('Value: %s' % value)
```

Output:

Key: key_1
Value: value_1

Key: key_3
Value: value_3

Key: key_2
Value: value_2

This works because the method `.items()` pulls all key-value pairs from a dictionary into a list of tuples:

```
my_dict = {'key_1': 'value_1',  
          'key_2': 'value_2',  
          'key_3': 'value_3',  
          }  
print(my_dict.items())
```

Output:

```
[('key_1', 'value_1'), ('key_3', 'value_3'), ('key_2', 'value_2')]
```

The syntax for `key, value in my_dict.items():` does the work of looping through this list of tuples, and pulling the first and second item from each tuple for us.

There is nothing special about any of these variable names, so Python code that uses this syntax becomes really readable. Rather than create a new example of this loop, let's just look at the original example again to see this in a meaningful context:

```
python_words = {'list': 'A collection of values that are not connected, but have an order.',  
                'dictionary': 'A collection of key-value pairs.',  
                'function': 'A named set of instructions that defines a set of actions in Python.',  
                }
```

```
for word, meaning in python_words.items():  
    print("\nWord: %s" % word)  
    print("Meaning: %s" % meaning)
```

Output:

Word: function

Meaning: A named set of instructions that defines a set of actions in Python.

Word: list

Meaning: A collection of values that are not connected, but have an order.

Word: dictionary

Meaning: A collection of key-value pairs.

Looping through all keys in a dictionary

Python provides a clear syntax for looping through just the keys in a dictionary:

```
my_dict = {'key_1': 'value_1',  
          'key_2': 'value_2',  
          'key_3': 'value_3',  
          }
```

```
for key in my_dict.keys():  
    print('Key: %s' % key)
```

Output:

```
Key: key_1  
Key: key_3  
Key: key_2
```

This is actually the default behavior of looping through the dictionary itself. So you can leave out the `.keys()` part, and get the exact same behavior:

```
my_dict = {'key_1': 'value_1',  
          'key_2': 'value_2',  
          'key_3': 'value_3',  
          }
```

```
for key in my_dict:  
    print('Key: %s' % key)
```

Output:

```
Key: key_1  
Key: key_3  
Key: key_2
```

The only advantage of using the `.keys()` in the code is a little bit of clarity. But anyone who knows Python reasonably well is going to recognize what the second version does. In the rest of our code, we will leave out the `.keys()` when we want this behavior.

You can pull out the value of any key that you are interested in within your loop, using the standard notation for accessing a dictionary value from a key:

```
my_dict = {'key_1': 'value_1',  
          'key_2': 'value_2',  
          'key_3': 'value_3',  
          }  
  
for key in my_dict:  
    print('Key: %s' % key)  
    if key == 'key_2':  
        print(" The value for key_2 is %s." % my_dict[key])
```

Output:

Key: key_1
Key: key_3
Key: key_2

The value for key_2 is value_2.

Let's show how we might use this in our Python words program. This kind of loop provides a straightforward way to show only the words in the dictionary:

```
python_words = {'list': 'A collection of values that are not connected, but have an order.',  
               'dictionary': 'A collection of key-value pairs.',  
               'function': 'A named set of instructions that defines a set of actions in Python',  
               }  
  
# Show the words that are currently in the dictionary.  
print("The following Python words have been defined:")  
for word in python_words:  
    print("- %s" % word)
```

Output:

The following Python words have been defined:
- function
- list
- dictionary

We can extend this slightly to make a program that lets you look up words. We first let the user choose a word. When the user has chosen a word, we get the meaning for that word, and display it:

```
python_words = {'list': 'A collection of values that are not connected, but have an order.',
                'dictionary': 'A collection of key-value pairs.',
                'function': 'A named set of instructions that defines a set of actions in Python',
                }

# Show the words that are currently in the dictionary.
print("The following Python words have been defined:")

for word in python_words:
    print("- %s" % word)

# Allow the user to choose a word, and then display the meaning for that word.
requested_word = raw_input("\nWhat word would you like to learn about? ")
print("\n%s: %s" % (requested_word, python_words[requested_word]))
```

Output:

The following Python words have been defined:

- function
- list
- dictionary

What word would you like to learn about? list

list: A collection of values that are not connected, but have an order.

This allows the user to select one word that has been defined. If we enclose the input part of the program in a while loop, the user can see as many definitions as they'd like:

```
python_words = {'list': 'A collection of values that are not connected, but have an order.',
                'dictionary': 'A collection of key-value pairs.',
                'function': 'A named set of instructions that defines a set of actions in Python',
                }

# Show the words that are currently in the dictionary.
print("The following Python words have been defined:")
for word in python_words:
    print("- %s" % word)

requested_word = ''
while requested_word != 'quit':
    # Allow the user to choose a word, and then display the meaning for that word.
    requested_word = raw_input("\nWhat word would you like to learn about? (or 'quit') ")
```

```
if requested_word in python_words.keys():
    print("\n %s: %s" % (requested_word, python_words[requested_word]))
else:
    # Handle misspellings, and words not yet stored.
    print("\n Sorry, I don't know that word.")
```

Output:

The following Python words have been defined:

- function
- list
- dictionary

What word would you like to learn about? (or 'quit') list

list: A collection of values that are not connected, but have an order.

What word would you like to learn about? (or 'quit') dictionary

dictionary: A collection of key-value pairs.

What word would you like to learn about? (or 'quit') quit

Sorry, I don't know that word.

This allows the user to ask for as many meanings as they want, but it takes the word "quit" as a requested word.

Let's add an `elif` clause to clean up this behavior:

```
python_words = {'list': 'A collection of values that are not connected, but have an order.',
                'dictionary': 'A collection of key-value pairs.',
                'function': 'A named set of instructions that defines a set of actions in Python.',
                }
```

```
# Show the words that are currently in the dictionary.
```

```
print("The following Python words have been defined:")
```

```
for word in python_words:
    print("- %s" % word)
```

```
requested_word = ''
```

```
while requested_word != 'quit':
```

```
    # Allow the user to choose a word, and then display the meaning for that word.
```

```
    requested_word = raw_input("\nWhat word would you like to learn about? (or 'quit') ")
```

```
    if requested_word in python_words.keys():
```

```
        # This is a word we know, so show the meaning.
```

```
        print("\n %s: %s" % (requested_word, python_words[requested_word]))
```

```
    elif requested_word != 'quit':
```

```
        # This is not in python_words, and it's not 'quit'.
```

```
        print("\n Sorry, I don't know that word.")
```

```
    else:
```



```
# The word is quit.  
print "\n Bye!"
```

Output:

The following Python words have been defined:

- function
- list
- dictionary

What word would you like to learn about? (or 'quit') function

function: A named set of instructions that defines a set of actions in Python.

What word would you like to learn about? (or 'quit') dictionary

dictionary: A collection of key-value pairs.

What word would you like to learn about? (or 'quit') list

list: A collection of values that are not connected, but have an order.

What word would you like to learn about? (or 'quit') class

Sorry, I don't know that word.

What word would you like to learn about? (or 'quit') quit

Bye!

Looping through all values in a dictionary

Python provides a straightforward syntax for looping through all the values in a dictionary, as well:

```
my_dict = {'key_1': 'value_1',  
          'key_2': 'value_2',  
          'key_3': 'value_3',  
          }  
for value in my_dict.values():  
    print('Value: %s' % value)
```

Output:

```
Value: value_1  
Value: value_3  
Value: value_2
```

We can use this loop syntax to have a little fun with the dictionary example, by making a little quiz program. The program will display a meaning, and ask the user to guess the word that matches that meaning. Let's start out by showing all the meanings in the dictionary:

```
python_words = {'list': 'A collection of values that are not connected, but have an order.',  
                'dictionary': 'A collection of key-value pairs.',  
                'function': 'A named set of instructions that defines a set of actions in Python.',  
                }  
  
for meaning in python_words.values():  
    print("Meaning: %s" % meaning)
```

Output:

```
Meaning: A named set of instructions that defines a set of actions in Python.  
Meaning: A collection of values that are not connected, but have an order.  
Meaning: A collection of key-value pairs.
```

Now we can add a prompt after each meaning, asking the user to guess the word:

```
python_words = {'list': 'A collection of values that are not connected, but have an order.',  
                'dictionary': 'A collection of key-value pairs.',  
                'function': 'A named set of instructions that defines a set of actions in Python.',  
                }  
  
# Print each meaning, one at a time, and ask the user  
# what word they think it is.  
for meaning in python_words.values():  
    print("\nMeaning: %s" % meaning)  
    guessed_word = raw_input("What word do you think this is? ")
```

```
# The guess is correct if the guessed word's meaning matches the current meaning.
if python_words[guessed_word] == meaning:
    print("You got it!")
else:
    print("Sorry, that's just not the right word.")
```

Output:

Meaning: A named set of instructions that defines a set of actions in Python.
What word do you think this is? function
You got it!

Meaning: A collection of values that are not connected, but have an order.
What word do you think this is? function
Sorry, that's just not the right word.

Meaning: A collection of key-value pairs.
What word do you think this is? dictionary
You got it!

This is starting to work, but we can see from the output that the user does not get the chance to take a second guess if they guess wrong for any meaning. We can use a while loop around the guessing code, to let the user guess until they get it right:

```
python_words = {'list': 'A collection of values that are not connected, but have an order.',
                'dictionary': 'A collection of key-value pairs.',
                'function': 'A named set of instructions that defines a set of actions in Python.',
                }

# Print each meaning, one at a time, and ask the user
# what word they think it is.
for meaning in python_words.values():
    print("\nMeaning: %s" % meaning)
    # Assume the guess is not correct; keep guessing until correct.
    correct = False
    while not correct:
        guessed_word = input("\nWhat word do you think this is? ")

        # The guess is correct if the guessed word's meaning matches the current meaning.

        if python_words[guessed_word] == meaning:
            print("You got it!")
            correct = True
        else:
            print("Sorry, that's just not the right word.")
```

Output:

Meaning: A named set of instructions that defines a set of actions in Python.

What word do you think this is? function

You got it!

Meaning: A collection of values that are not connected, but have an order.

What word do you think this is? dictionary

Sorry, that's just not the right word.

What word do you think this is? list

You got it!

Meaning: A collection of key-value pairs.

What word do you think this is? dictionary

You got it!

This is better. Now, if the guess is incorrect, the user is caught in a loop that they can only exit by guessing correctly. The final revision to this code is to show the user a list of words to choose from when they are asked to guess:

```
python_words = {'list': 'A collection of values that are not connected, but have an order.',
                'dictionary': 'A collection of key-value pairs.',
                'function': 'A named set of instructions that defines a set of actions in Python.',
                }

def show_words(python_words):
    # A simple function to show the words in the dictionary.
    display_message = ""
    for word in python_words.keys():
        display_message += word + ' '
    print display_message

    # Print each meaning, one at a time, and ask the user
    # what word they think it is.
    for meaning in python_words.values():
        print("\n%s" % meaning)
        # Assume the guess is not correct; keep guessing until correct.
        correct = False
        while not correct:
            print("\nWhat word do you think this is?")
            show_words(python_words)
            guessed_word = raw_input("- ")

            # The guess is correct if the guessed word's meaning matches the current meaning.
            if python_words[guessed_word] == meaning:
                print("You got it!")
```

```
        correct = True
    else:
        print("Sorry, that's just not the right word.")
```

Output:

A named set of instructions that defines a set of actions in Python.

```
What word do you think this is?
function list dictionary
- function
You got it!
```

A collection of values that are not connected, but have an order.

```
What word do you think this is?
function list dictionary
- dictionary
Sorry, that's just not the right word.
```

```
What word do you think this is?
function list dictionary
- list
You got it!
```

A collection of key-value pairs.

```
What word do you think this is?
function list dictionary
- dictionary
You got it!
```

Looping through a dictionary in order

Dictionaries are quite useful because they allow bits of information to be connected. One of the problems with dictionaries, however, is that they are not stored in any particular order. When you retrieve all of the keys or values in your dictionary, you can't be sure what order you will get them back. There is a quick and easy way to do this, however, when you want them in a particular order.

Let's take a look at the order that results from a simple call to *dictionary.keys()*:

```
python_words = {'list': 'A collection of values that are not connected, but have an order.',
                'dictionary': 'A collection of key-value pairs.',
                'function': 'A named set of instructions that defines a set of actions in Python.',
                }
for word in python_words.keys():
    print(word)
```

Output:

```
function
list
dictionary
```

The resulting list is not in order. The list of keys can be put in order by passing the list into the *sorted()* function, in the line that initiates the for loop:

```
python_words = {'list': 'A collection of values that are not connected, but have an order.',
                'dictionary': 'A collection of key-value pairs.',
                'function': 'A named set of instructions that defines a set of actions in Python.',
                }
for word in sorted(python_words.keys()):
    print(word)
```

Output:

```
dictionary
function
list
```

This approach can be used to work with the keys and values in order. For example, the words and meanings can be printed in alphabetical order by word:

```
python_words = {'list': 'A collection of values that are not connected, but have an order.',
                'dictionary': 'A collection of key-value pairs.',
                'function': 'A named set of instructions that defines a set of actions in Python.',
                }
for word in sorted(python_words.keys()):
    print("%s: %s" % (word.title(), python_words[word]))
```

Output:

Dictionary: A collection of key-value pairs.

Function: A named set of instructions that defines a set of actions in Python.

List: A collection of values that are not connected, but have an order.

In this example, the keys have been put into alphabetical order in the for loop only; Python has not changed the way the dictionary is stored at all. So the next time the dictionary is accessed, the keys could be returned in any order. There is no way to permanently specify an order for the items in an ordinary dictionary, but if you want to do this you can use the **OrderedDict** structure.

Exercises

- Write a Python program to remove duplicates from Dictionary
Hint: Create your own dictionary with duplicate values.
Loop through dictionary and store key and value in new dictionary except duplicates
print new dictionary
- Write a Python program to match key values in two dictionaries.
Sample dictionary: {'key1': 1, 'key2': 3, 'key3': 2}, {'key1': 1, 'key2': 2}
Expected output: key1: 1 is present in both x and y



Motivate to Innovate...



**STEM
BUILDERS**
Learning Center

MATH | ROBOTICS | TECHNOLOGY | EVENTS | CAMPS



contactus@stembuilders.com



www.stembuilders.com